عمر، نبداو من الصفر

ShopMate AI • دليل الفهم بالدارجة • التحضير للمناقشة ديال 22 أكتوبر 2026

هاد الدليل علاش؟

الهدف هو تفهم شنو بنيتي، علاش كل جزء كاين، وكيفاش الأجزاء كيخدمو مع بعضياتهم. ماشي ضروري تحفظ الكود كامل اليوم. خاصك تبني صورة واضحة، ومن بعد تربطها بالملفات.

كيفاش تقراه؟

قرا درس واحد. سد الدليل. شرح الفكرة بصوتك باستعمال مثال. إلا تعطلتي، رجع غير للفقرة اللي ما فهمتيش. كل درس فيه سؤال صغير؛ الأجوبة كاينة فالآخر.

المثال اللي غادي نتبعو

زبون بغا يشري جوج تيشورتات. فالمثال فقط: الثمن 120 درهم للوحدة والمخزون 5. هادو أرقام تعليمية، ماشي بيانات حقيقية من المتجر ديالك.

شنو راجعنا؟

الشرح مبني على README وملفات الكود المحلية ديال المشروع نهار 20 أكتوبر 2026. ماشي شهادة أن النشر خدام أو أن جميع الاختبارات دازت. الكود يقدر يتبدل من بعد.

أول فكرة تحفظها بالفهم

تطبيق ديال البيع بالدفع عند التسليم. الزبون يقدر يشوف المنتجات ويسول المساعد ShopMate ويطلب. المسؤول كيسير الكاتالوغ والوثائق وكيشوف أثر تفاعلات المساعد.

خريطة القراية

ما تقراش كلشي دفعة وحدة. هاد المراحل مترتبة باش كل فكرة تبني على اللي قبلها.

المرحلة 1 • الأساس

الصفحات 3 حتى 8: شنو هو برنامج، API، backend، frontend، وكيفاش كتقرا الكود وكتلقى الملفات.

المرحلة 2 • التطبيق بلا ذكاء اصطناعي

الصفحات 9 حتى 14: قاعدة البيانات، تسجيل الدخول، الصلاحيات، الكاتالوغ والطلبات.

المرحلة 3 • الذكاء الاصطناعي

الصفحات 15 حتى 22: RAG، LLM، الأدوات، التأكيد، الحماية، SSE والسجلات.

المرحلة 4 • الربط والتشغيل

الصفحات 23 حتى 26: Docker، n8n، MCP، النشر والاختبارات.

المرحلة 5 • المناقشة

الصفحات 27 حتى 31: رحلة كاملة، الحدود الحالية، أسئلة اللجنة، خطة المراجعة وأجوبة التمارين.

كيفاش تعرف أنك فهمتي؟

إلا قدرت تجاوب على هادو بلا حفظ: شكون كيرسل الطلب؟ شكون كيتحقق؟ فين كتتخزن المعلومة؟ شنو يقع إلا فشل جزء؟ راه بديتي كتفهم المشروع فعلاً.

1 • شنو هو البرنامج؟

البرنامج هو تعليمات مكتوبة للحاسوب باش يدير خدمة.

Variable • متغير

بحال علبة مسمية كتحتافظ بقيمة. مثلاً stock فيه العدد المتوفر. إلا كانت القيمة 5 وبعنا 2، القيمة الجديدة تولي 3.

Function • دالة

مجموعة تعليمات عندها اسم. كتستقبل معلومات، كتدير خدمة، وقد ترجع نتيجة. checkStock كتقلب على المخزون ديال منتوج.

Array و Object

Array كيجمع خصائص ديال حاجة وحدة: منتوج عندو اسم وثمن ومخزون Object هي لائحة: مثلاً لائحة المنتجات اللي فالطلب.

Condition • شرط

if كتعني: إلا تحقق الشرط، دير هاد الخدمة. إلا كانت الكمية المطلوبة أكبر من المخزون، نرفض الطلب.

Folder و File

الملف فيه كود أو وثيقة. المجلد كيجمع ملفات مرتبطة. تقسيم المشروع كيساعدنا نلقاو المسؤولية ديال كل جزء.

سؤال 1

إلا stock فيه 5 والزبون طلب 7، شنو خاص البرنامج يدير؟ علاش؟

2 • ثلاث طبقات: بحال محل

تخيل متجر: الواجهة اللي كيشوف الزبون، الموظف اللي كيسير، والدفتر اللي فيه البيانات.

Frontend • الواجهة

هو اللي كيبان فالتليفون: أزرار، صور، فورموليرات، شات. فمشروعك هو mobile باستعمال React Native وExpo. كيجمع اختيار الزبون وكيعرض النتيجة.

Backend • الخادم

هو البرنامج اللي كيستقبل الطلبات، كيتحقق من الهوية والحقوق، كيحسب وكيقرا ويكتب فالقاعدة. فمشروعك هو backend باستعمال Node.js وExpress.

Database • قاعدة البيانات

هي التخزين المنظم والدائم: مستخدمين، منتجات، مخزون، طلبات ورسائل. PostgreSQL هي قاعدة البيانات ديالك.

```
Téléphone → API Express → PostgreSQL
Téléphone ← Réponse JSON ← API Express
```

علاش ما يخدمش التليفون مباشرة مع القاعدة؟

حيت المستخدم يقدر يبدل الطلبات اللي كيرسل. قواعد الأمان والثمن والمخزون خاصها تبقى فالخادم. إخفاء زر فالواجهة بوحدو ما كيحميش البيانات.

سؤال 2

إلا الزبون بدل ثمن المنتوج فالتليفون، واش الخادم خاصو يقبل داك الثمن؟

mobile/app • backend/src/app.ts • backend/src/config/database.ts

3 • شنو كيدير كل اسم تقني؟

ما تحفظش الأسماء بوحدها. ربط كل اسم بالخدمة ديالو.

TypeScript و JavaScript

JavaScript لغة البرمجة. TypeScript كتزيد وصف أنواع البيانات وكتكشف أخطاء أثناء التحليل أو البناء. ولكن البيانات اللي جاية من الشبكة خاصها حتى تحقق وقت التشغيل.

Express و Node.js

Express كينظم المسارات: مثلاً شنو نديرو ملي يجي طلب ديال المنتجات. JavaScript يخدم فالخادم Node.js كيخلي كود.

Expo و React Native

Expo كيبني واجهة الموبايل بمكونات React Native. كيوفر أدوات التشغيل والبناء وخدمات للموبايل. Expo Router كيربط الملفات بالشاشات والتنقل.

SecureStore و Axios و Zustand

Zustand كينظم حالة مشتركة فالواجهة، بحال المستخدم الحالي. Axios كيرسل طلبات HTTP. SecureStore كيتستعمل لتخزين حساس فالموبايل؛ ماشي قاعدة بيانات المنتجات.

Zod و Sequelize

Sequelize هو ORM: كيربط كود TypeScript بجداول SQL. Zod كيتأكد أن المدخلات عندها الشكل ما كتوافقش AGENTS.md؛ الإشارة القديمة لـSequelize فـPrisma والقيم المقبولة. الكود الحالي كيستعمل هاد التنفيذ.

pgvector و PostgreSQL

PostgreSQL كتحفظ البيانات. pgvector إضافة كتخزن وتقارن المتجهات العددية المستعملة فالبحث الدلالي.

backend/package.json • mobile/package.json • backend/src/models

4 • كيفاش كيهضر التليفون مع الخادم؟

API هي واجهة متفق عليها بين برنامجين. HTTP هو البروتوكول المستعمل هنا.

Request • الطلب

فيه العنوان، الطريقة، headers وممكن body. العنوان كيحدد الخدمة. headers فيها معلومات بحال نوع المحتوى ووسيلة إثبات الهوية.

Methods • الطرق

GET للقراية. POST للإرسال أو الإنشاء. PUT/PATCH للتعديل حسب المسار. DELETE للحذف. كل مسار عندو قواعدو.

JSON • شكل البيانات

صيغة نصية كتجمع مفاتيح وقيم. مثال تعليمي ديال منتوج: الاسم والثمن والمخزون. JSON ماشي قاعدة بيانات وماشي لغة ذكاء اصطناعي.

```
GET /api/products
Authorization: Bearer <access-token>

{"name":"T-shirt","price":120,"stock":5}
```

Response • الجواب

200 نجاح؛ 201 إنشاء؛ 400 مدخلات غير مقبولة؛ 401 خاص المصادقة أو التوكن غير صالح؛ 403 ما عندكش الحق؛ 404 ما تلقاش المورد؛ 429 طلبات كثيرة؛ 500 خطأ فالخادم. الشكل الدقيق ديال الجواب كيختلف حسب المسار.

سؤال 3

شنو الفرق بين 401 و403؟ شرح بمثال ديال زبون ومسؤول.

backend/src/app.ts • backend/src/routes

5 • كيفاش تقرا سطر كود؟

هاد المثال مبسط للتعليم، ماشي نسخ حرفي من المشروع.

```
const quantity = 2;
const product = await Product.findOne({
  where: { id: productId, storeId }
});
if (!product) throw new Error("Not found");
if (product.stock < quantity) {
  throw new Error("Insufficient stock");
}
```

const

كنسمي قيمة باش نرجع نستعملها. quantity هي الكمية المطلوبة.

await

كنستنا نتيجة خدمة غير فورية، بحال القراية من قاعدة البيانات. كتستعمل داخل دالة async. ما كتقصدش نحبس الخادم كامل.

where و findOne

قلب على سجل واحد كيوافق الشروط: معرف المنتوج ومعرف المتجر. هادي طريقة ORM؛ من تحت كاين استعلام SQL.

if و throw

كنختابرو واش المنتوج موجود وواش المخزون كافي. throw كيرسل خطأ لطبقة معالجة الأخطاء. فعملية شراء حقيقية كنحتاجو حتى transaction وlock.

طريقة القراية

بدا بالمدخلات، من بعد الشروط، من بعد الوصول للقاعدة، ومن بعد النتيجة. ما تحاولش تفهم جميع imports قبل ما تعرف الهدف ديال الدالة.

6 • فين تلقى كل حاجة؟

كل مرة كتشرح ميزة، حاول تورّي الملف المسؤول عليها.

backend/src/app.ts و server.ts

app كيركب middlewares والمسارات. server كيربط مع قاعدة البيانات وكيبدا يسمع للطلبات.

backend/src/routes

فيه auth وproducts وcategories وorders وdocuments وRoute. agent هي نقطة دخول HTTP، كتستقبل وتوجه الخدمة.

backend/src/models و schemas

models كتوصف السجلات والعلاقات. schemas كتوصف المدخلات المقبولة. Model ديال Product ماشي هو schema ديال طلب إنشاء Product.

backend/src/services

orderConfirmation للتأكيد؛ agentTools للأدوات؛ llm للتواصل مع النموذج؛ rag للبحث فالوثائق؛ guardrails للحماية؛ mcpShipmentClient للتتبع الخارجي.

mobile/app و mobile/src

app تخزين الحالة، المصادقة والمكونات المشتركة. ملفات API، فيه اتصال src. فيه الشاشات والتنقل (tabs) كتنظم تبويبات الواجهة.

migrations و docs و stitch_shopmate

migrations كتغير بنية القاعدة. docs فيها وثائق المشروع. stitch_shopmate مرجع التصميم؛ التصميم المرئي بوحدو ما كيبرهنش أن الخدمة مربوطة بالخادم.

7 • قاعدة البيانات: شنو كتخزن؟

Table جدول، Row سجل، Column خاصية، ID معرف كيخلينا نميزو السجل.

Store / User / Customer

Store هو المتجر. User حساب الدخول والأدوار. Customer بيانات الزبون المستعملة فالطلبات. ماشي بالضرورة نفس المعرف Customerو User فالمسارات كاين ربط بالمتجر والبريد؛

Category / Product

Category كتجمع المنتجات فصنف. Product فيه بيانات المنتوج بحال الثمن والمخزون ومعرف المتجر والصنف. imageUrl رابط صورة، ماشي بالضرورة خدمة رفع وتخزين الصور.

Order / OrderItem

Order رأس الطلب: صاحبو والحالة والمجموع. OrderItem سطر فيه المنتوج والكمية وثمن الوحدة وقت الشراء. طلب واحد يقدر يكون فيه بزاف ديال السطور.

Conversation / Message / AgentLog

Conversation محادثة، Message رسالة داخلها. AgentLog أثر تقني لتفاعل: الأدوات والمصادر والمدة والاستهلاك والنتيجة.

Document / Chunk / RefreshToken

Document وثيقة معرفة. Chunk جزء منها مع تمثيل عددي. RefreshToken سجل لتجديد الدخول؛ القيمة المخزنة هي hash ديال التوكن.

```
Store → Products / Customers / Orders
Customer → Orders → OrderItems → Product
Conversation → Messages
Document → Chunks
```

backend/src/models/index.ts • backend/src/models/*.ts

8 • التسجيل والدخول

Authentication كتجاوب: شكون نتا؟ Authorization كتجاوب: شنو مسموح ليك تدير؟

التسجيل

الواجهة كترسل المعلومات. الخادم كيتحقق من الشكل. كيحسب hash لكلمة السر باستعمال bcrypt وكيحفظ الحساب. الحساب العادي كيتسجل بدور CUSTOMER.

ماشي تشفير قابل للعكس Hash

الخادم ما خاصوش يرجع كلمة السر الأصلية من hash. وقت الدخول bcrypt كيقارن الكلمة المدخلة مع hash المخزن بطريقة مناسبة.

الدخول

الخادم كيلقى الحساب وكيقارن كلمة السر. إلا نجح التحقق، كيرجع access token وrefresh token. إلا فشل، ما كيعطيش صلاحية الدخول.

بطاقة موقعة • JWT

Access token كيحمل معلومات بحال userId وstoreId وrole. التوقيع كيخلي الخادم يكشف التلاعب. الموقع ماشي بالضرورة مشفر؛ ما نحطوش فيه أسرار JWT.

```
Login → Vérification bcrypt → Tokens
Requête → Vérification JWT → Contrôle du rôle → Service
```

سؤال 4

علاش ما نحفظوش كلمة السر واضحة فالقاعدة؟ وعلاش ما نثقوش فـrole جاية من body؟

backend/src/routes/auth.ts • backend/src/config/jwt.ts • backend/src/middlewares/auth.ts

9 • Refresh rotation و logout

Access token للاستعمال اليومي القصير، وrefresh token لتجديد الدخول بلا إعادة كلمة السر كل مرة.

عندما كيسالي access token

الواجهة كتلقى 401، وكتطلب refresh. الخادم كيتحقق من refresh token ومن الصلاحية ديالو، وكيصدر زوج جديد ديال التوكنات.

Rotation • التبديل

القديم كيتحيد وكيخرج واحد جديد. الهدف هو ما يبقاش نفس السر صالح للتجديد طول الوقت. Refresh فالقاعدة كيكون hash بدل القيمة الأصلية.

Axios mutex

إلا عدة طلبات جابو 401 فنفس الوقت، آلية فالعميل كتنسق التجديد باش ما يبداوش كاملين refresh مستقل فنفس اللحظة.

Logout • الخروج

الواجهة كتحيد معلومات الدخول والخادم كيلغي refresh. ولكن access JWT اللي خرج من قبل يقدر يبقى صالح حتى يسالي الوقت ديالو إلا ما كانتش آلية إبطال إضافية.

حد خاص تعرفو

الـrotation الحالية ما خاصناش نوصفوها بضمان مطلق ضد جميع الطلبات المتزامنة. حذف القديم وإصدار الجديد خاصهم مراجعة التزامن والمعاملات باش يكون الضمان أقوى.

سؤال 5

واش logout كيعني آلياً أن أي access token قديم ولى باطل فوراً؟

backend/src/routes/auth.ts • mobile/src/api/client.ts • mobile/src/auth

10 • Multi-tenant: كل متجر وبياناتو

Tenant هنا هو متجر. نفس التطبيق يقدر يخدم أكثر من متجر مع عزل البيانات.

storeId

معرف المتجر كيدخل فشروط الوصول للبيانات التابعة ليه. فالمسارات المحمية كيجي من JWT المتحقق منو فالخادم، ماشي من اختيار الزبون فالطلب.

مثال

متجر A عندو منتوج، ومتجر B عندو منتوج آخر. إلا مستخدم A عرف ID ديال منتوج B، الخادم خاصو يبحث بالـID وstoreId معاً. معرفة المعرف ما كتمنحش الحق.

الزبون داخل نفس المتجر

حتى داخل A، ما خاصش زبون يشوف طلب زبون آخر. كنحتاجو التحقق من مالك الطلب زيادة على المتجر.

الدور ADMIN

إظهار شاشة الإدارة غير للمسؤول مفيد فالواجهة. لكن الحماية الحقيقية كتكون حتى فالخادم قبل القراءة أو التعديل.

تفصيل فنسخة الديمو

التسجيل والدخول كيرتبطو بمتجر الديمو المحدد فالخادم. بعض قراءات الكاتالوغ العامة كتستعمل متجر الديمو كذلك. ما تقولش إن كل request عامة كتحتاج JWT، ولا إن التسجيل كيدير اختيار حر للمتجر.

سؤال 6

علاش الفلترة بالـorderId بوحدو ما كافياش لحماية طلبات الزبناء؟

backend/src/middlewares/tenant.ts • backend/src/models/tenantScope.ts • backend/src/routes/orders.ts

11 • الكاتالوغ والإدارة

CRUD: Create إنشاء، Read قراية، Update تعديل، Delete حذف.

رحلة عرض المنتجات

الشاشة كتطلب المنتجات. الخادم كيتحقق من الفلاتر والمتجر. القاعدة كترجع النتائج. الواجهة كتبيّن اللائحة، التحميل، الفراغ أو الخطأ.

تقسيم اللائحة • Pagination

ما كنجيبووش جميع المنتجات دفعة وحدة. كنطلبو جزء، ومن بعد جزء آخر. هادشي كيخفف الاستجابة والذاكرة.

المسؤول كيعدل

يقدر يدير عمليات الكاتالوغ المسموحة: منتجات، أصناف، ثمن ومخزون. الخادم كيتحقق من الدور ADMIN والمدخلات. حذف منتوج خاصو واجهة تأكيد ومعالجة القيود المرتبطة بالطلبات.

التحقق • Validation

مثلاً الكمية عدد موجب والثمن قيمة مقبولة. TypeScript بوحدو ما كيكفيش، حيت الطلب الخارجي يقدر يكون بأي شكل. Zod كيتحقق وقت التشغيل.

واش التغيير كيبان مباشرة؟

القاعدة كتكون المرجع الجديد. المساعد اللي كيستعمل أدوات القراية كيلقى القيم الحالية. الشاشة المفتوحة قد تحتاج إعادة تحميل؛ هادشي ما كيعنيش كاين broadcast تلقائي لجميع الهواتف.

سؤال 7

إلا المسؤول بدل الثمن، واش خاصنا نعاودو ندربو LLM باش يعرفو؟

backend/src/routes/products.ts • backend/src/routes/categories.ts • mobile/app/(tabs)

12 • الطلب والمخزون و Transaction

بيع واحد خاصو يحدث الطلب والسطور والمخزون بشكل منسجم.

مثال بالحساب

المخزون 5، الزبون باغي 2، ثمن الوحدة 120. مجموع السطور 240 والمخزون من بعد 3. الثمن كيتقرا من القاعدة؛ ما كنخليوش الزبون يفرض المجموع.

Transaction • معاملة

كتجمع تغييرات مترابطة: إما تنجح كلها ويتدار commit، وإما يتدار rollback ويرجع أثر المعاملة كما كان. ما نبغيوش ينقص المخزون والطلب ما يتسجلش.

Lock • قفل على السجل

إلا جوج زبناء بغاو آخر قطعة فنفس الوقت، القفل كينظم تعديل السجل. العملية الثانية كتستنى وتعاود التحقق من المخزون المتاح.

OrderItem كيحفظ الثمن

إلا ثمن المنتوج تبدل غداً، خاص الطلب القديم يبقى عندو ثمن الشراء ديالو. لهذا كنحتاجو ثمن الوحدة فسطر الطلب.

التأكيد

المسار ديال الشات كيحضّر recap ومن بعد كينفذ عند التأكيد. المسار HTTP ديال الطلب عندو schemas وقواعد خاصة؛ ما نخلطوش بين الآليتين ونفترضو أنهما نفس الدالة.

سؤال 8

شنو المشكل إلا نقصنا المخزون ثم وقع خطأ قبل إنشاء الطلب بلا transaction؟

backend/src/routes/orders.ts • backend/src/services/orderConfirmation.ts • backend/src/models/OrderItem.ts

13 • LLM و Agent: شنو الفرق؟

LLM نموذج لغوي كيعالج النص وكيولد كلام. Agent هو النظام اللي كينظم استعمال هاد النموذج مع أدوات وقواعد.

ماشي قاعدة بيانات المتجر LLM

النموذج يقدر يعطي كلام مقنع حتى إلا ما عندوش المعلومة. ما عندوش تلقائياً المخزون الحقيقي ولا طلبات الزبناء.

دور Agent

الخادم كيجمع السؤال والتاريخ والسياق، كيطبق الحماية، كيطلب من النموذج، كيتحقق من الأدوات المقترحة، كينفذ المسموح وكيحضّر الجواب.

System prompt

تعليمات على الدور واللغة والحدود. مفيدة، ولكن ما كتعوضش التحقق البرمجي من الهوية والحقوق والمدخلات.

Provider

مزود خارجي للنموذج. الكود كيقبل إعدادات endpoint/model/key عبر البيئة. فالمناقشة قول المزود اللي ضبطتي فعلاً؛ ما تدعيش أن جميع المزودين مجربين.

Tokens

قطع من النص كيستعملها النموذج. ماشي دائماً كلمات كاملة. الاستهلاك فيه نص مرسل ونص مولد، وكيأثر على التكلفة والمدة.

سؤال 9

شكون كيقرر واش أداة معينة مسموحة وتتنفذ فعلاً: النموذج بوحدو ولا الخادم؟

backend/src/services/llm.ts • backend/src/routes/agent.ts

14 • RAG: نجبدو المعرفة قبل الجواب

RAG = Retrieval Augmented Generation. بالبساطة: نقلبو على معلومة مناسبة، ونمدوها للنموذج باش يجاوب منها.

المشكل

الزبون كيسول على مدة التوصيل. النموذج ما خاصوش يخترع سياسة المتجر. خاصو يعتمد على وثيقة حقيقية مرفوعة من المسؤول.

مرحلتان

هي البحث على المقاطع المناسبة وقت Retrieval. هي إدخال الوثائق وتحضيرها للبحث Ingestion السؤال. Generation هي صياغة الجواب من السياق.

رفع الوثيقة

المسؤول كيرفع ملف md أو txt حتى 2 Mo عبر POST /api/documents. الخادم كيحضّر المقاطع والمتجهات ويحفظها. واجهة الإدارة كتخدم لائحة الوثائق والحذف وحالة المعالجة؛ إلا فشل الرفع ما نفترضوش أن المعرفة تزادت.

مثال تعليمي

وثيقة كتقول: التوصيل للمدينة بين يومين وثلاثة. السؤال: شحال حتى توصلني؟ كنرجعو المقطع المناسب ونطلبو جواب مبني عليه مع المصدر.

```
Document → Chunks → Embeddings → pgvector
Question → Embedding → Recherche → Contexte → Réponse + source
```

واش هادي كتدرب النموذج؟

لا. فهاد المشروع كنضيفو سياق وقت الطلب، ما كنبدلوش أوزان النموذج. رفع وثيقة ماشي fine-tuning.

سؤال 10

شنو الفرق بين نرفع سياسة إرجاع جديدة وبين ندرب نموذج جديد؟

backend/src/services/rag.ts • backend/src/routes/documents.ts

15 • Chunk و Embedding و Similarité

هاد الكلمات كتشرح كيفاش البحث كيلقى معنى قريب حتى إلا الكلمات ماشي هي نفسها.

Chunk • مقطع

بدل ما نرسلو الوثيقة كاملة، كنقسموها لمقاطع. فالكود الهدف تقريباً 800 حرف مع overlap ديال 100 حرف. التداخل كيحافظ على سياق قريب من الحدود.

Embedding • تمثيل عددي

النص كيتحول للائحة أعداد كتستعمل لقياس القرب الدلالي. فالإعداد الافتراضي: text-embedding-3-small و1536 بُعد. ماشي لائحة ديال 1536 كلمة.

مثال للفهم فقط

تصور أن كل نص نقطة فخريطة. جمل قريبة فالمعنى كتكون أقرب حسب النموذج. الخريطة الحقيقية فيها أبعاد كثيرة؛ رسم بجوج أبعاد غير تشبيه.

HNSW و Cosine

كيسرع البحث على جيران قريبين بطريقة تقريبية. HNSW index. مقياس مرتبط باتجاه المتجهات Cosine. ما كيولدش الجواب، غير كيساعد فالاسترجاع.

Threshold • العتبة

الإعداد الافتراضي فالبحث 0.75 وtopK ديال 6. إلا ما كاينش سياق مناسب، الجواب المعرفي يكون: Je n'ai pas cette information. 0.75 ماشي ضمان صحة بنسبة 75%.

سؤال 11

علاش خاص سؤال المستخدم والوثائق يستعملو embeddings متوافقين؟

backend/src/services/rag.ts • backend/sql

16 • RAG ولا Tools؟

هاد الفرق هو قلب المشروع، وغالباً غادي تسولك عليه اللجنة.

المعرفة الثابتة نسبياً

سياسة الإرجاع، شروط التوصيل، معلومات المتجر: كنبحثو عليها فالوثائق باستعمال RAG ونبينو المصدر.

البيانات الحية

الثمن والمخزون وحالة الطلب: أدوات كتقرا من قاعدة البيانات أو الخدمة المخصصة. ما نعتمدوش على وثيقة قديمة باش نقولو شحال بقا فالمخزون.

الأفعال

إنشاء أو إلغاء طلب: أدوات كتجهز العملية وكتطلب التأكيد، ومن بعد التنفيذ المحمي. الجواب النصي بوحدو ما كيخلقش طلب.

أمثلة

«شنو شروط الإرجاع؟» → RAG.

«واش باقي التيشورت؟» → checkStock.

«بغيت جوج» → createOrder ثم recap ثم تأكيد.

«فين وصلات؟» → getShipmentStatus بعد التحقق من الملكية.

الأدوات ماشي غير كتابة

أداة ولكن ما كتبدل والو فالقاعدة. الفرق searchProducts. كيشمل حتى القراية Function Calling العملي هو مصدر المعلومة وطبيعة الخدمة.

سؤال 12

علاش ما نحطوش الثمن فوثيقة RAG ونخليو المساعد يعتمد عليه دائماً؟

backend/src/services/agentTools.ts • backend/src/services/rag.ts

17 • الأدوات الثمانية وكيفاش كتتنفذ

النموذج كيقترح اسم أداة وarguments. الخادم كيتحقق قبل التنفيذ.

أدوات الكاتالوغ

كتقرا checkStock. كتجيب تفاصيل منتوج getProductDetails. كتقلب على المنتجات searchProducts التوفر والمعطيات الحية المرتبطة به.

أدوات الطلبات

كتجيب حالة طلب getOrderStatus. كتبدأ مسار إنشاء طلب بالتأكيد createOrder. كتجيب طلبات الزبون listMyOrders. كتبدأ مسار الإلغاء بالتأكيد cancelOrder.

أداة التتبع

كتطلب حالة الشحنة عبر MCP getShipmentStatus بعد التحقق من الطلب والمتجر والزبون.

```
{"name":"checkStock","arguments":{"productId":42}}
```

سلسلة التحقق

اسم داخل allowlist → قراءة JSON → تحقق Zod → حقن هوية المستخدم والمتجر من الخادم → تنفيذ الخدمة. النموذج ما كيختارش هوية زبون آخر.

السقف

الحد هو 3 استدعاءات أدوات فالدور الواحد. هادشي كيمنع حلقات مكلفة بلا نهاية. Allowlist كتحصر القدرات؛ ما كايناش أداة تمنح خصم عشوائي.

backend/src/services/agentTools.ts • backend/src/schemas/agentTools.ts

18 • التأكيد قبل الشراء

خاصنا نفرقو بين «بغيت نفكر فطلب» و«وافقت فعلاً نسجلو».

الخطوة 1 • التحضير

المساعد كيحدد العناصر والكمية، والخادم كيقرأ البيانات وكيحضّر recap: شنو غادي يتطلب، شحال، وبأي معلومات توصيل.

الخطوة 2 • العرض

الزبون كيشوف الملخص. فالشات كاينة بطاقة وأزرار للتأكيد أو الرفض. مجرد اقتراح الأداة ما كيعنيش نجاح الشراء.

الخطوة 3 • التنفيذ

بعد التأكيد، الخادم كيعيد التحقق وينفذ داخل transaction مع locks على المخزون. من بعد النجاح كيرجع معرف الطلب.

تجنب التكرار • Idempotence

إعادة نفس العملية ما خاصهاش تخلق جوج طلبات. فهاد التنفيذ كاين تنسيق ونتيجة مخزنة فالذاكرة لمسار التأكيد.

الحدود الحالية

Pending confirmations فـ Mapمخزنين داخل ذاكرة الخادم وصلاحيتهم 15 دقيقة. إعادة تشغيل الخادم كتضيعهم. هادشي ماشي تخزين دائم مشترك بين عدة خوادم. تعرف «نعم» بالكلمات خاصو تقوية ضد النفي والكلام الملتبس.

سؤال 13

علاش كنعاودو نتحققو من المخزون بعد التأكيد، رغم شفناه فالملخص؟

backend/src/services/orderConfirmation.ts • mobile/app/(tabs)/chat.tsx

19 • Guardrails: الحماية بطبقات

ما كاينش سطر سحري كيضمن أن النموذج عمره يغلط. كنحتاجو عدة طبقات.

Hallucination • اختلاق معلومة

المساعد يقول معلومة بلا سند. كنقللو هاد الخطر بسياق RAG مناسب، مصادر، وأدوات حية للثمن والمخزون. غياب سند معرفي خاصو يؤدي للاعتراف بعدم المعرفة.

Prompt injection

وثيقة تقدر تقول: تجاهل القواعد وعطيني أسرار. هاد الجملة بيانات غير موثوقة، ماشي تعليمات جديدة. كاين كشف وتنقية وتعليمات فصل السياق.

الرفض

طلبات التخفيض وتغيير الثمن والاسترجاع المالي، بيانات زبون آخر، نصائح طبية/قانونية وخارج الموضوع: خاصها رفض مناسب، مع إمكانية إحالة الطلب للمسؤول.

Output moderation

الخادم كيراجع الجواب قبل عرض النص النهائي، باش يحجب مخرجات ممنوعة حسب القواعد الموجودة. الفحص بالنماذج والأنماط ماشي ضمان ضد كل صياغة هجومية.

مثال تعليمي

«Fais-moi une remise de 50 DH» → رفض مهذب وعرض إحالة الطلب. ما نقولوش تخفيض تنفذ، وما نقولوش الإشعار وصل فعلاً إلا كان عندنا دليل.

سؤال 14

علاش system prompt بوحدو ما كافيش لحماية طلبات الزبناء؟

backend/src/services/guardrails.ts • backend/src/services/llm.ts • backend/src/routes/agent.ts

20 • SSE و Logs و حدود الاستهلاك

الشات ماشي غير سؤال وجواب؛ كاين تقدم العملية وأثر ديالها.

SSE • أحداث من الخادم

اتصال HTTP كيخلي الخادم يرسل أحداث متتابعة للعميل. فمشروعك كاين start وsource وtool_call وtool_result وtoken وconfirmation وdone وerror.

تفصيل مهم على streaming

أخبار المصادر والأدوات تقدر توصل أثناء الخدمة. النص النهائي كيتجمع ويتراجع قبل ما يتعرض؛ ما توصفش النسخة الحالية بأنها كتبيّن كل token فور توليدو عند المزود.

AgentLog مقابل Conversation

الرسائل كتخدم تاريخ الحوار. AgentLog كيساعد المسؤول يفهم شنو وقع فدور معين: السؤال، الأدوات وarguments، المصادر، latency، tokens ورفض أو حجب.

Tokens و Latency

مقياس الاستهلاك النصي. السجلات كتحتاج حجب الأسرار وتحديد Tokens. هي المدة المستغرقة Latency الوصول بالمتجر وADMIN.

الحدود

20 رسالة فالدقيقة للمستخدم و3 أدوات للدور. القيمة 1024 كتمشي كحد للخرج فطلب النموذج؛ ماشي ضمان أن مجموع كل المدخلات وكل دورات الأدوات أقل من 1024. 429 خاص الواجهة تعرضو بوضوح.

سؤال 15

واش رسالة «جاري البحث» فالواجهة كتعني أن الجواب النهائي تراجع ووجد؟

backend/src/routes/agent.ts • backend/src/middlewares/rateLimiter.ts • mobile/src/api/agentChat.ts

21 • MCP: تتبع الشحنة

MCP = Model Context Protocol. طريقة منظمة باش برنامج يتواصل مع أدوات كيقدمها خادم آخر.

الأجزاء

عندك خادم API الرئيسي، client ديال MCP فالـbackend، وخادم MCP منفصل كيعرض getShipmentStatus.

الرحلة

الزبون يسول فين وصل الطلب. API كيتحقق أن الطلب ديالو ومن متجره، من بعد client كيتصل بـMCP_SERVER_URL. النتيجة كتترجع للمساعد باش يشرحها.

```
Client → Agent → Vérification de propriété
Agent → MCP client → MCP server → Statut simulé
Agent → Réponse au client
```

بيانات محاكاة • Mock

فالديمو بيانات النقل محاكاة. هادشي كيبين الربط والبروتوكول، ولكن ما كيثبتش أن شركة توصيل حقيقية مرتبطة.

fallback و Timeout

العميل عندو مهلة 5 ثوان وكيحقق فالجواب. إلا الخدمة ما جاوباتش، يرجع رسالة مفهومة بدل تفاصيل داخلية أو حالة شحنة مخترعة.

سؤال 16

شنو الفرق بين getOrderStatus من قاعدة المتجر وgetShipmentStatus من خدمة التتبع؟

backend/src/mcp/shipmentServer.ts • backend/src/services/mcpShipmentClient.ts

22 • n8n و Webhook

n8n كينظم أتمتة خارج التطبيق. Webhook طلب HTTP كيخبر خدمة أخرى أن حدث وقع.

الحدث

تخلق طلب أو المساعد أحال طلب للمسؤول. Backend كيجهز تفاصيل الحدث وكيبعثها للرابط المضبوط فالإعدادات.

سلسلة خطوات • Workflow

يستقبل → يقرأ نوع الحدث → يحضر نص الإشعار → قناة إشعار للمسؤول. إعداد القناة Webhook والمفاتيح كيبقى مطلوباً خارج الكود.

```
Order created → Backend webhook → n8n → Notification
```

علاش نفصلوه؟

قواعد الشراء تبقى فالخادم. الإشعارات يمكن تبديل قناتها وإعدادها فالأتمتة بلا خلطها مع حساب المخزون.

حدود الديمو

وجود ملف workflow ما كيعنيش أنه منشور ومفعل. فشل الإشعار ما خاصوش يتحول لوعد أن المسؤول توصل. خاص تجربة حقيقية وإثبات وصول الإشعار.

سؤال 17

إلا n8n كان طافي، واش هادشي بالضرورة كيعني الطلب ما تسجلش؟

backend/src/services/n8nNotifications.ts • backend/n8n

23 • Docker و migrations و seed

الهدف هو تجهيز تشغيل متشابه فالأجهزة والخادم.

Image و Container

عملية تشغيل معزولة نسبياً انطلقت من Container. وصف مبني فيه التطبيق وما يحتاجو Image container. Volume كيحتافظ ببيانات القاعدة خارج عمر image.

Multi-stage Dockerfile

مرحلة build كتجهز الكود، ومرحلة runtime كتحتافظ بما يلزم للتشغيل. مستخدم non-root كيقلل الصلاحيات داخل الحاوية.

Compose

كيجمع API وPostgreSQL مع pgvector وشبكة وإعدادات وفحوص صحة. داخل حاوية API، اسم خدمة DB هو اللي كيشير للقاعدة؛ localhost كيعود لنفس الحاوية.

Migration

تغيير منظم فبنية القاعدة: جدول جديد أو عمود أو index. ماشي نسخة احتياطية. pgvector SQL كيمكن الامتداد والبنية اللازمة للبحث المتجهي حسب السكريبتات.

Seed

كيضيف بيانات البداية أو الديمو. تشغيله فالإنتاج خاص يكون مقصود ومراجع؛ ماشي كل إعادة تشغيل خاصها تعاود تزيد بيانات تجريبية.

Environment

الإعدادات بحال DB host ومفاتيح API كتجي من البيئة. env. ما يتنشرش فـ.env.example. git فيه أسماء وقيم وهمية. ما تحطش مفتاح المزود السري فـEXPO_PUBLIC.

backend/Dockerfile • docker-compose.yml • backend/migrations • backend/src/seed.ts

24 • النشر والاختبارات

«الكود موجود» و«جربناه فالإنتاج» جوج درجات مختلفين ديال الدليل.

Deployment • النشر

تشغيل API وقاعدة البيانات فخدمة مستضافة، إعداد secrets وHTTPS وتشغيل migrations. الموبايل خاصو عنوان backend المنشور فإعداداته.

health و CORS

CORS كينظم طلبات المتصفح من origins تطبيق وحدو وما كيحميش ؛ ماشي بديل للمصادقة native. health endpoint كيبين واش الخدمة كتجاوب، وماشي بالضرورة فحص لكل اعتماد خارجي.

Unit test

كيختبر جزء صغير: مثلاً schema كيرفض كمية سالبة. Integration test كيختبر أجزاء مرتبطة. E2E كيدوز رحلة مستخدم كاملة.

Mock

بديل مبرمج لخدمة حقيقية أثناء الاختبار. مفيد باش نختبرو منطق محدد بسرعة؛ ما كيبرهنش أن PostgreSQL الحقيقي أو الشبكة خدامين.

شنو كاين هنا؟

backend كيبني TypeScript وكيشغل سكريبتات اختبارات للمصادقة والعزل والأدوات فال npm test ولا تحققنا من هاتف suite فهاد جلسة إعداد الدليل ما شغلناش n8n. والتأكيد والحماية والتكلفة و حقيقي.

سؤال 18

واش test ناجح باستعمال mock كافي باش نقولو إن البيع المتزامن فقاعدة الإنتاج مضمون؟

backend/package.json • backend/src/scripts • mobile/src/config/api.ts

25 • دوز الرحلة كاملة بصوتك

هادي حكاية المشروع اللي خاصك تقدر تشرحها فالسبورة.

1 • دخول

الزبون يسجل الدخول. API كيتحقق من كلمة السر وكيصدر التوكنات. الواجهة كتستعمل access token فالطلبات المحمية.

2 • سؤال معرفي

يسول على الإرجاع. API يسترجع مقاطع وثائق المتجر المناسبة، ويطلب جواب مبني عليها. إلا ما لقا سند مناسب، يصرح بعدم المعرفة.

3 • منتوج

يسول على تيشورت. النموذج يقترح searchProducts أو checkStock. الخادم يتحقق من الأداة وargumentsويقرا البيانات الحية من متجر المستخدم.

4 • شراء

يبغي جوج. كيتعرض recap. يوافق. الخادم يعاود يتحقق ويكتب الطلب والسطور وينقص المخزون داخل transaction.

5 • من بعد

الواجهة كتبين رقم الطلب. إشعار n8n ممكن يتبعث إذا الربط مهيأ. التتبع عبر MCP كيرجع حالة محاكاة فالديمو. ADMIN يراجع أثر الشات فـAgentLog.

التمرين الكبير

رسم هاد الرحلة بوحدك وسمّي فين كنستعملو JWT وRAG وTools وtransaction وMCP. إلا قدرت تفسر علاش كل واحد كاين، راه فهمتي العمود الفقري للمشروع.

26 • شنو تقول بصراحة على الحدود؟

الفهم القوي كيبان حتى ملي كتفرق بين الهدف، التنفيذ الحالي، والدليل اللي عندك.

ما تقولش: أمان مطلق

قول: عندي طبقات تحقق من الهوية والمتجر والملكية والمدخلات، ومعاها guardrails واختبارات. الكشف بالأنماط ما كيضمنش صد جميع الهجمات.

ما تقولش: logout كيلغي كلشي فوراً

قول: كنبطل refresh ونحيد التوكنات من العميل؛ access JWT قد يبقى صالح حتى نهاية المدة.

ما تقولش: تأكيد دائم وموزع

قول: تأكيد الشات مخزن فالذاكرة حالياً بمهلة، ونحتاج تخزين مشترك ودائم باش نقوي التشغيل بعد restart أو مع عدة instances.

ما تقولش: 75% احتمال صحة

قول: هادي عتبة similarity فالبحث، خاص معايرتها بأسئلة ممثلة ومراجعة الأجوبة والمصادر.

ما تقولش: كل التكاملات حقيقية

قول: MCP كيرجع بيانات محاكاة، وn8n خاصو إعداد وتجربة، والنشر والهاتف كيحتاجو أدلة تشغيل مستقلة.

README والكود

فيه بعض عبارات أقوى من الضمان الحالي، بحال الإبطال الفوري أو منع التكرار بشكل مطلق. README شرحك يتبع التنفيذ وحدوده، وتحديث التوثيق يكون فمهمة منفصلة.

27 • أسئلة اللجنة: جواب وفهم

هادو صيغ قصيرة بالفرنسية. فهم المعنى بالدارجة قبل ما تستعملهم.

Présente ton projet.

ShopMate est une application mobile e-commerce COD avec un assistant. Il répond à partir des documents de la boutique et utilise des outils contrôlés pour les données et les commandes.

المعنى: تطبيق بيع بمساعد عندو مصادر وحدود، ماشي شات عام.

Pourquoi RAG et function calling ?

Le RAG fournit le contexte documentaire. Les outils lisent les données actuelles et exécutent les opérations autorisées.

المعنى: سياسة التوصيل من الوثيقة، المخزون من القاعدة، والشراء عملية محمية.

Comment isoles-tu les boutiques ?

Le serveur utilise le storeId de l'identité authentifiée et filtre les accès. Pour les commandes, il vérifie aussi le client propriétaire.

المعنى: معرف المتجر بوحدو ما كافيش لملكية الطلب.

Comment évites-tu une commande involontaire ?

Le chat prépare un récapitulatif. Après confirmation, le serveur vérifie les données et exécute la transaction.

المعنى: نية الشراء ماشي هي الموافقة النهائية.

As-tu utilisé l'IA pour coder ?

Oui. Je vérifie le code produit, je documente les corrections et je dois pouvoir expliquer les choix et les limites.

ما تدعيش تحقق ما درتيهش. ورّي مثال حقيقي من journal ديالك.

28 • خطة يومين بلا تشتت

المناقشة اللي قلتي نهار 22 أكتوبر. الموعد العام فالوثائق 24 أكتوبر؛ حضر على أساس الأقرب.

20 أكتوبر • الأساس

ساعة: الصفحات 3 حتى 10 مع رسم المعمارية. راحة. ساعة: الكاتالوغ والطلبات والمعاملات. نصف ساعة: حل الأسئلة بصوتك بلا دليل.

21 أكتوبر • الذكاء الاصطناعي

ساعة ونصف: RAG والأدوات والتأكيد. راحة. ساعة: الحماية وSSE وMCP وDocker. سجل شرح صوتي قصير وسمع فين كتخلط.

21 أكتوبر • ديمو

راجع رحلة الدخول، البحث، سؤال وثيقة، recap، رفض ثم تأكيد، عرض الطلب، ثم الإدارة والوثائق والسجلات. استعمل بيانات ديمو وخلي نتيجة كل خطوة واضحة.

بديل الديمو

وجد تسجيل فيديو ولقطات من تجربة فعلية إلا الشبكة تعطل. إلا عرضتي تسجيلاً، قول بأنه تسجيل؛ ما تقدموش كتجربة مباشرة.

22 أكتوبر • مراجعة خفيفة

كرر تعريف المشروع والرحلة الكاملة وخمس نقاط: RAG، storeId، JWT مقابل الأدوات، التأكيد، transaction. إلا ما عرفتيش تفصيل، قول فين غادي تقلب عليه.

ديالك Journal

كتب بكلامك: شنو طلبت فهاد الجلسة، شنو فهمت، شنو الحدود اللي اكتشفت، وشنو باقي ما واضحش. ما تسجلش نتيجة اختبار ما شغلتوش.

29 • أجوبة التمارين ومفتاح المراجعة

حاول تجاوب قبل ما تقرا. الهدف تفهم السبب، ماشي تحفظ نفس الجملة.

1 حتى 3

1: نرفض كمية 7 لأن المتوفر 5. 2: لا؛ الثمن المرجعي من الخادم والقاعدة. 3: 401 هوية ناقصة أو غير صالحة؛ 403 هوية معروفة ولكن بلا الصلاحية المطلوبة.

4 حتى 6

4: hash و الموقع قد يبقى صالح access، فالطلب يقدر يتزور. 5: لا roleكيقلل ضرر تسرب كلمات السر، حتى انتهاء المدة. 6: خاص حتى المتجر وصاحب الطلب.

7 حتى 9

7: لا، الأداة كتقرا الثمن الحالي. 8: المخزون ينقص بلا طلب؛ transaction كتجنب الأثر الجزئي داخلها. 9: الخادم هو اللي يتحقق وينفذ، النموذج كيقترح.

10 حتى 12

10: RAG كيضيف سياق وقت السؤال، التدريب كيغير النموذج. 11: باش المقارنة العددية تكون عندها معنى وأبعاد متوافقة. 12: الثمن فالوثيقة يقدر يولي قديم.

13 حتى 15

13: المخزون يقدر يتبدل بين الملخص والتأكيد. 14: الحقوق خاصها تحقق برمجي مستقل عن النص. 15: لا، حدث التقدم ماشي الجواب النهائي المراجع.

16 حتى 18

16: حالة الطلب الداخلية مقابل تتبع الشحنة بالخدمة الخارجية. 17: لا بالضرورة؛ الإشعار منفصل عن تسجيل الطلب. 18: لا؛ خاص تجربة مناسبة بقاعدة حقيقية وتزامن.

من بعد هاد الدليل

فتح ملف واحد، اختار دالة وحدة، وشرح: المدخلات، التحقق، القراءة أو الكتابة، النتيجة، وحالة الفشل. هادي هي الطريقة باش توصل للفهم التفصيلي سطر بسطر.